ISM3232 — Week 12 Lab

OOP III — Applied Practice & Design — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 12 · Unit 3 · Object-Oriented Design

Contents

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 12 Lab — OOP III: Applied Practice & Design
Unit	Unit 3 · Object-Oriented Design
Class length	Full class period (75 minutes)
Format	Independent design-and-build, gated by an instructor design review checkpoint
Prerequisites	Weeks 10–11: `BusinessRequest`/`RequestManager` — two entity classes, a manager class, composition, boundary/independence testing
Student-facing lab page	Week 12 In-Class Lab — Module 6, "OOP III: Applied Practice and Design"
Parts covered	Part 1 (design document, enforced) – Part 5 (simulation + ritual)
Submission	`design.md` + 2 screenshots, GitHub URL, Canvas, completion credit

This lab is structurally different from every prior week: **students design and build their own multi-class system, in a business domain of their own choosing** — not the `BusinessRequest` template from Weeks 10–11. The lab page's own rule is worth enforcing exactly as written, without softening it: **students may not open VS Code to write class code until their `design.md` is reviewed by the instructor.** This means today's central facilitation challenge isn't teaching new syntax — it's running an efficient, fair design-review queue for an entire room, and this guide's Part 1 is built specifically around that logistics problem. There is no single shared example to live-code; instead, this guide walks a complete worked example (a different domain — event registrations — from the `BusinessRequest` template) that you can demonstrate as *process*, not as an answer key for students to copy.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Design a multi-class OOP system on paper *before* writing code — naming classes, attributes, methods, and business rules in advance.
2. Independently build two related entity classes and a manager class, applying every pattern from Weeks 10–11 (constructors, instance methods, `__repr__`, composition) to a novel domain.

3. Write and pass at least six `pytest` tests covering initial state, business logic, a boundary case, and manager-level behavior including instance independence.
4. Build a `main.py` simulation exercising the full system, and run the established submission ritual.

3 Before Class — Setup Checklist

☐ **Build your own complete worked example before class** — this guide's model system (`Event`, `Registration`, `RegistrationManager`) is fully verified below; walking through it yourself once means you can demonstrate the *design-to-code* process live without solving any specific student's actual project for them.

☐ Decide your design-review approval bar in advance, and write it somewhere visible (the board, a shared doc) before Part 1 begins — see the "What 'approved' means" callout in Part 1; without a clear, fast, consistent bar, the review queue becomes this lab's single biggest bottleneck.

☐ Plan your physical/virtual queue mechanism before class — a raised-hand line, a shared sign-up doc, or calling rows in sequence — anything that avoids a chaotic scrum of 20+ students all raising hands at once around the same time, which is a very likely failure mode if this isn't planned for explicitly.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module06_oop/` venv and files from Weeks 10–11
- Students: their `module06_oop/` project, plus a genuine business domain idea of their own (encourage choosing this *before* class if your course structure allows a heads-up)

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: "design before code, enforced"	4
0:04–0:24	Part 1 — Design document (write + instructor review)	20
0:24–0:36	Part 2 — Build Entity Class 1	12
0:36–0:50	Part 3 — Build Entity Class 2 and the Manager	14
0:50–1:03	Part 4 — Six or more pytest tests	13

Time	Segment	Minutes
1:03–1:11	Part 5 — Main simulation and ritual	8
1:11–1:15	Wrap-up, submission checklist	4

Part 1 is deliberately given the most time of any part this semester, and for a genuinely structural reason: it includes both independent writing time *and* a queued instructor review that gates all further progress — a student who finishes their design in 8 minutes may still wait several more for their turn in the review queue, and that wait needs to be built into the plan, not treated as slack.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

> *"Today you design and build your own system — your own business domain, your own classes, not the purchase-request template from the last two weeks. And I'm enforcing something new: you cannot open a code editor until I've reviewed your design document. This isn't busywork — designing before coding is a genuinely professional habit, and today is the one day this semester built specifically to practice it, with real consequences for skipping it."*

Do: Write the day's gate rule on the board, exactly: **No code until `design.md` is reviewed. Raise your hand when it's done.**

6.2 Part 1 — Design Document (0:04–0:24, 20 min)

Teaching goal: A complete `design.md`, written entirely before any code — forcing students to think through class structure, attributes, methods, and business rules in the abstract, the way a real system would be planned before implementation begins.

Say to the class:

> *"Pick a business domain — something genuinely different from purchase requests. A library checkout system, event registrations, a food delivery order tracker, a gym membership manager — anything with two related kinds of things and rules that govern them. Fill in every field in this template. I will not review a design with blanks left in it."*

Do:

```
cd ~/ism3232/module06_oop && touch design.md && code design.md
```

Have students fill in every field:

```
# OOP Lab 12 Design
# Author: [Your Name]

## Business Domain
[Describe the system you are building in 1–2 sentences]

## Entity Class 1
Class name: ___
Attributes (name + type):
  - ___: ___
```

```
  - ___: ___
Methods (name + what it does + return type):
  - ___: ___ -> ___
  - ___: ___ -> ___

## Entity Class 2
Class name: ___
Attributes:
  - ___: ___
Methods:
  - ___: ___ -> ___

## Manager Class
Class name: ___
Holds: list of [entity class 1]
Methods:
  - add(item): adds to the list
  - list_by_status(status): filters by status
  - total(): returns sum of a numeric attribute
  - report(): prints formatted summary

## Business Rules (will become methods and test cases)
1. ___
2. ___
3. ___

## Out of Scope
[What you are NOT building]
```

Demonstrate your own worked example, live, as a model of the *process*, not as an answer to copy — this guide uses event registrations throughout:

```
## Business Domain
A system for managing registrations to a company's annual gala event —
tracking who's signed up, at what price tier, and whether they're
confirmed, pending, or cancelled.

## Entity Class 1
Class name: Registration
Attributes:
  - reg_id: int
  - attendee_name: str
```

```
  - event: Event (a reference to Entity Class 2)
  - price: float
  - status: str (defaults to 'Pending')
Methods:
  - confirm(): marks status as Confirmed -> None
  - cancel(): marks status as Cancelled -> None
  - is_vip(): returns True if price exceeds $500 -> bool

## Entity Class 2
Class name: Event
Attributes:
  - event_id: int
  - name: str
  - capacity: int
Methods:
  - (none required beyond __init__ and __repr__ for this domain)

## Manager Class
Class name: RegistrationManager
Holds: list of Registration
Methods:
  - add(registration)
  - list_by_status(status)
  - total(): sums price across all registrations
  - report(): prints confirmed/pending counts and total revenue

## Business Rules
1. A new registration starts as 'Pending' until confirmed.
2. A registration is VIP if its price exceeds $500.
3. Cancelled registrations should still count in total revenue history,
   but should not appear in the "Confirmed" report section.

## Out of Scope
Payment processing, waitlists, refunds, multi-event registrations.
```

Facilitation notes on running the review queue — this is the actual logistics challenge of Part 1:

- **What "approved" means — set a fast, consistent bar, and say it aloud before students start writing:** every field is filled in with real content (no blanks, no "TBD"); the two entity classes are genuinely different kinds of things, not the same concept renamed; at least one business rule is specific enough to become a real boundary-case test (a numeric threshold,

a status transition rule). A review should take well under a minute per student if the bar is fast and consistent — don't debate design quality in depth here, just confirm the document is complete and coherent enough to build from.

- **Queue mechanism:** call students up in a fixed order (by row, alphabetically, a sign-up sheet) rather than an open "raise your hand whenever" free-for-all, which tends to cluster chaotically around the 10–12 minute mark once most of the room finishes around the same time.
- **What to do with students who finish and get approved early:** they may begin Part 2 immediately — there's no reason to hold fast finishers back while the queue clears for everyone else; today's parts are independent enough that early starters simply move ahead at their own pace.
- **A design that's "too big" or "too small":** if a student's design has, say, six entity classes and elaborate business rules — redirect toward simplifying to exactly the required two entity classes plus manager, since Part 2–5's remaining 55 minutes assume that scope. If a design is too thin (e.g., an entity with only two attributes and no real business rule), push for at least one genuine, testable rule before approving — this is exactly what Part 4's required boundary test depends on existing.

Common student mistakes to watch for:

- Choosing two entity classes that are really the same concept with different names (e.g., "Customer" and "Client" as separate classes with near-identical fields) — redirect toward a genuine relationship instead, like this guide's `Registration` (has-a) `Event`.
- Business rules that are too vague to become a test ("the system should work well") — push for a specific, numeric, checkable rule, modeling the "VIP if price exceeds $500" example.
- Skipping ahead to write code before approval, out of impatience — enforce the rule as stated; this is the entire point of the exercise, and inconsistent enforcement undermines it for the whole room.

6.3 Part 2 — Build Entity Class 1 (0:24–0:36, 12 min)

Teaching goal: Independently build the first entity class exactly as designed — applying Week 10's `__init__`/methods/`__repr__` pattern to a student's own domain, with instructor circulation rather than lockstep live-coding.

Say to the class:

> *"Once you're approved, build Entity Class 1 exactly as you designed it. I'm circulating, not lecturing — this is where your own design becomes real code."*

State the requirements explicitly, matching the lab page:

- `__init__` with **at least 4 attributes**, including a status field defaulting to a pending-like value
- **At least 2 business logic methods that return values** — not `print()`, echoing Week 7's rule
- `__repr__` showing key attributes
- **Docstrings on every method**

Demonstrate your own worked example's Entity Class 1, as a model:

```
class Registration:
    """Represents one attendee's registration for an event."""

    def __init__(self, reg_id, attendee_name, event, price):
        self.reg_id = reg_id
        self.attendee_name = attendee_name
        self.event = event
        self.price = price
        self.status = 'Pending'

    def confirm(self):
        """Mark this registration as confirmed."""
        self.status = 'Confirmed'

    def cancel(self):
        """Mark this registration as cancelled."""
        self.status = 'Cancelled'

    def is_vip(self):
        """Return True if this registration's price exceeds $500."""
        return self.price > 500

    def __repr__(self):
```

```
        return f'Registration({self.reg_id}, {self.attendee_name}, {self.event.name}, ${self
```

Point out explicitly, as you circulate — this is worth checking on every student's version individually, not just demonstrating once:

- **The four-attribute minimum is easy to satisfy accidentally without a genuine status field** — confirm the status attribute specifically exists and defaults sensibly (`'Pending'`, `'Open'`, `'Draft'` — whatever fits the student's domain), since Part 4's required boundary/state tests depend on it.
- **"Business logic methods that return values"** — walk the room checking for stray `print()` calls inside methods, the exact Week 7 violation; a method like `confirm()` that prints instead of setting `self.status` will silently break Part 3's manager-level filtering later, since `list_by_status()` depends on `self.status` actually being set.
- `__repr__` referencing `self.event.name` in this guide's example — worth noting explicitly if a student's Entity Class 1 similarly references an attribute of a not-yet-built Entity Class 2: this is fine, and even expected, given the domains are meant to relate — but the `event` parameter itself (an actual `Event` object, not yet defined at this point in the file) means Entity Class 1's `__init__` may reference a class that technically doesn't exist yet in the file. This is a good moment to note: Python doesn't require class definitions to appear in dependency order within a file the way some students might expect — what matters is that both classes exist by the time an actual object is *created*, in `main.py`, not the order they're *defined* in `models.py`.

Common student mistakes to watch for:

- Fewer than 4 real attributes — padding with a trivial, unused field to hit the count rather than genuinely modeling the domain; a quick "does this attribute get used anywhere later" question surfaces this.
- Business logic methods that take no meaningful action (e.g., a method that returns a hard-coded value regardless of the object's actual state) — redirect toward methods that genuinely read or modify `self`'s own attributes.

6.4 Part 3 — Build Entity Class 2 and the Manager (0:36–0:50, 14 min)

Teaching goal: A second entity class that genuinely relates to the first, plus a manager class following Week 11's exact composition pattern, applied independently.

Say to the class:

> *"Entity Class 2 should be a different kind of thing that relates to Entity Class 1 in your domain — not a duplicate. Then the manager, following exactly the same shape as last week's `RequestManager`: holds a list, adds to it, filters it, totals it, reports on it."*

State the manager requirements explicitly, matching the lab page:

- `__init__` with an empty list
- `add(item)` method
- `list_by_status(status)` that filters and returns a list
- `total()` that sums a numeric attribute
- `report()` that prints a formatted summary

Demonstrate your own worked example's Entity Class 2 and Manager, as a model:

```
class Event:
    """Represents a single event that people can register for."""

    def __init__(self, event_id, name, capacity):
        self.event_id = event_id
        self.name = name
        self.capacity = capacity

    def __repr__(self):
        return f'Event({self.event_id}, {self.name}, capacity={self.capacity})'


class RegistrationManager:
    """Manages a collection of event registrations."""

    def __init__(self):
        self.registrations = []

    def add(self, registration):
        """Add a Registration to the collection."""
        self.registrations.append(registration)

    def list_by_status(self, status):
```

```
        """Return registrations matching the given status."""
        return [r for r in self.registrations if r.status == status]

    def total(self):
        """Return the total revenue across all registrations."""
        return sum(r.price for r in self.registrations)

    def report(self):
        """Print a formatted summary of all registrations."""
        confirmed = self.list_by_status('Confirmed')
        pending = self.list_by_status('Pending')
        print(f'Total registrations: {len(self.registrations)} | Revenue: ${self.total():,.2
        print(f'Confirmed: {len(confirmed)} | Pending: {len(pending)}')
        print('Confirmed registrations:')
        for r in confirmed:
            print(f'  {r}')
```

Point out explicitly, as you circulate:

- **The relationship between the two entity classes is worth checking individually per student** — in this guide's example, `Event` doesn't hold or reference `Registration` at all; instead, each `Registration` holds a *reference to* the `Event` it's for (`self.event = event`, set in `Registration.__init__`). This is a common, correct shape (a "many registrations, one event" relationship) — but not the only valid one; some student domains may genuinely need the reverse (a class holding a list of a *different* related class, not the manager's own list). Both are fine — the point is the relationship should reflect something *true about the domain*, not be arbitrary.
- **`list_by_status(status)` must be genuinely parameterized** — some students, echoing Week 11's `RequestManager.get_by_status` too literally, may hardcode a specific status inside the method instead of using the `status` parameter; check this specifically, since Part 4's tests will call it with different status values and expect different results each time.
- **`total()` summing "a numeric attribute"** — confirm the student has identified which of their entity's attributes is the numeric one worth summing (a price, a quantity, a duration) — this should be a genuine, meaningful business total in their domain, not an arbitrary numeric field chosen just to satisfy the requirement.

Common student mistakes to watch for:

- A manager `add(item)` method that doesn't actually append to the list (e.g., reassigning instead of appending, silently discarding prior additions) — a good "does the count actually grow with each add" spot-check.
- `report()` calling `print()` — correctly, this time, unlike the entity classes' business-logic methods; make sure students don't over-apply "no print in business logic" to `report()` itself,

which is *explicitly* a presentation method whose entire job is to print.

6.5 Part 4 — Six or More pytest Tests (0:50–1:03, 13 min)

Teaching goal: A complete test suite covering the required categories — initial state, business logic, a boundary case, and manager-level behavior including independence — applied to each student's own system.

Say to the class:

> *"Six tests, minimum, and I want every one of these six categories represented — not six variations on the same check."*

State the required coverage explicitly, matching the lab page:

- Default status / initial state of a new instance
- Each business logic method (at least 2 methods tested)
- At least one boundary/edge case
- Manager: `add` increases count
- Manager: filter returns correct subset
- Manager: two manager instances are independent

Demonstrate your own worked example's tests, as a model — nine here, one more than the six-minimum, showing genuine coverage across all required categories:

```python
# tests/test_models.py
from models import Registration, Event, RegistrationManager

event = Event(1, 'Annual Gala', 200)

def test_default_status_is_pending():
    reg = Registration(1, 'Test', event, 100)
    assert reg.status == 'Pending'

def test_confirm_changes_status():
    reg = Registration(2, 'Test', event, 100)
    reg.confirm()
    assert reg.status == 'Confirmed'

def test_cancel_changes_status():
    reg = Registration(3, 'Test', event, 100)
    reg.cancel()
    assert reg.status == 'Cancelled'

def test_is_vip_true():
    reg = Registration(4, 'Test', event, 750)
```

```
    assert reg.is_vip() is True

def test_is_vip_false():
    reg = Registration(5, 'Test', event, 300)
    assert reg.is_vip() is False

def test_is_vip_boundary():
    reg = Registration(6, 'Test', event, 500)
    assert reg.is_vip() is False  # > not >=

def test_manager_add_increases_count():
    mgr = RegistrationManager()
    mgr.add(Registration(7, 'Test', event, 100))
    assert len(mgr.registrations) == 1

def test_manager_filter_by_status():
    mgr = RegistrationManager()
    r1 = Registration(8, 'A', event, 100)
    r2 = Registration(9, 'B', event, 200)
    mgr.add(r1); mgr.add(r2)
    r1.confirm()
    confirmed = mgr.list_by_status('Confirmed')
    assert len(confirmed) == 1
    assert confirmed[0].attendee_name == 'A'

def test_managers_are_independent():
    mgr1 = RegistrationManager()
    mgr2 = RegistrationManager()
    mgr1.add(Registration(10, 'A', event, 100))
    assert len(mgr2.registrations) == 0
```

Run it:

```
pytest -v
```

Verified output — all nine pass:

```
tests/test_models.py::test_default_status_is_pending PASSED
tests/test_models.py::test_confirm_changes_status PASSED
tests/test_models.py::test_cancel_changes_status PASSED
tests/test_models.py::test_is_vip_true PASSED
tests/test_models.py::test_is_vip_false PASSED
tests/test_models.py::test_is_vip_boundary PASSED
tests/test_models.py::test_manager_add_increases_count PASSED
```

```
tests/test_models.py::test_manager_filter_by_status PASSED
tests/test_models.py::test_managers_are_independent PASSED
9 passed
```

Facilitation notes, since every student's actual test content will differ:

- **Check for genuine category coverage, not just a count of six** — a student with six tests that are all variations of "does `confirm()` work" has technically hit the number but missed the point; walk the room specifically asking students to point out which test satisfies which of the six required categories.
- **The boundary case is the one most likely to be missing or weak** — by now (three labs into boundary-testing practice, following Weeks 7–8's precedent), most students should reach for this instinctively, but it's worth an explicit spot-check per student, since it's the single most-skipped required category historically in comparable exercises.
- **The independence test is structurally identical across every domain** — two managers, add to one, assert the other's collection is empty — this is worth confirming is present in something close to that exact shape, since it's the most mechanically transferable of the six requirements from Weeks 10–11's templates.

Common student mistakes to watch for: Same failure patterns as Weeks 10–11's tests (testing the same object twice under different variable names rather than genuinely two independent objects; a boundary test using >= reasoning when the actual method uses >, or vice versa) — walk the room checking these specifically, since they're proven, recurring mistake patterns from the prior two labs' equivalent exercises.

6.6 Part 5 — Main Simulation and Ritual (1:03–1:11, 8 min)

Teaching goal: A `main.py` exercising the complete, independently-designed system end to end, followed by the now-fully-automatic submission ritual.

Say to the class:

> *"Last step: a simulation showing your whole system working together, then the ritual, exactly as every week since Week 4."*

State the requirements explicitly, matching the lab page:

- Creates a manager instance
- Adds at least 4 entity instances
- Calls at least one status-changing method
- Calls `report()` to print the summary

Demonstrate your own worked example's `main.py`, as a model:

```python
# main.py
from models import Registration, Event, RegistrationManager

annual_gala = Event(1, 'Annual Gala', 200)

mgr = RegistrationManager()
mgr.add(Registration(1, 'Taylor', annual_gala, 750))
mgr.add(Registration(2, 'Jordan', annual_gala, 150))
mgr.add(Registration(3, 'Morgan', annual_gala, 300))
mgr.add(Registration(4, 'Riley', annual_gala, 500))

mgr.registrations[0].confirm()
mgr.registrations[1].confirm()

mgr.report()
```

Run it. Verified output:

```
Total registrations: 4 | Revenue: $1,700.00
Confirmed: 2 | Pending: 2
Confirmed registrations:
  Registration(1, Taylor, Annual Gala, $750, Confirmed)
  Registration(2, Jordan, Annual Gala, $150, Confirmed)
```

Now the full ritual, unchanged from every prior week:

```
ruff format . && ruff check . && pytest -v
```

```
git add . && git commit –m 'lab 12: OOP III applied practice' && git push
```

Common student mistakes to watch for:

- Fewer than 4 entity instances added — a quick visual count during circulation.
- No status-changing method called at all, so `report()`'s output shows everything still in its default state — technically satisfies "calls report()" but misses the more meaningful requirement of demonstrating the system's actual behavior; encourage at least one status change before the report.

7 Wrap-Up (last ~4 minutes)

Review the submission checklist together:

- ☐ `design.md` completed and instructor-approved *before* any code was written
- ☐ Entity Class 1: `__init__` with 4+ attributes including status, 2+ returning business logic methods, `__repr__`, docstrings on every method
- ☐ Entity Class 2: genuinely related to Entity Class 1, not a duplicate concept
- ☐ Manager class: `add`, `list_by_status`, `total`, `report`, all functioning correctly
- ☐ `tests/test_models.py`: 6+ tests covering all required categories, all passing
- ☐ `main.py`: manager created, 4+ entities added, at least one status change, `report()` called
- ☐ Git commit made, with a message including "lab 12"
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 13: "Today's design document — naming classes, attributes, and relationships before writing any code — is exactly the skill Week 13 formalizes into real database design: the same entities, attributes, and relationships, expressed as SQL tables instead of Python classes."

8 Appendix A — Full Worked Example (`models.py` + `main.py` + `tests/test_models.py`)

A complete, verified reference system — event registrations — distinct from the `BusinessRequest` template, demonstrating the full required shape. Use this to calibrate what an approved design and a complete submission should look like; do not distribute it as a copyable answer.

```python
# models.py
# Author: [Instructor demo]

class Event:
    """Represents a single event that people can register for."""

    def __init__(self, event_id, name, capacity):
        self.event_id = event_id
        self.name = name
        self.capacity = capacity

    def __repr__(self):
        return f'Event({self.event_id}, {self.name}, capacity={self.capacity})'


class Registration:
```

```
    """Represents one attendee's registration for an event."""

    def __init__(self, reg_id, attendee_name, event, price):
        self.reg_id = reg_id
        self.attendee_name = attendee_name
        self.event = event
        self.price = price
        self.status = 'Pending'

    def confirm(self):
        """Mark this registration as confirmed."""
        self.status = 'Confirmed'

    def cancel(self):
        """Mark this registration as cancelled."""
        self.status = 'Cancelled'

    def is_vip(self):
        """Return True if this registration's price exceeds $500."""
        return self.price > 500

    def __repr__(self):
        return f'Registration({self.reg_id}, {self.attendee_name}, {self.event.name}, ${self


class RegistrationManager:
    """Manages a collection of event registrations."""

    def __init__(self):
        self.registrations = []

    def add(self, registration):
        """Add a Registration to the collection."""
        self.registrations.append(registration)

    def list_by_status(self, status):
        """Return registrations matching the given status."""
        return [r for r in self.registrations if r.status == status]

    def total(self):
        """Return the total revenue across all registrations."""
```

```
        return sum(r.price for r in self.registrations)

    def report(self):
        """Print a formatted summary of all registrations."""
        confirmed = self.list_by_status('Confirmed')
        pending = self.list_by_status('Pending')
        print(f'Total registrations: {len(self.registrations)} | Revenue: ${self.total():,.2
        print(f'Confirmed: {len(confirmed)} | Pending: {len(pending)}')
        print('Confirmed registrations:')
        for r in confirmed:
            print(f'  {r}')
```

```
# main.py
from models import Registration, Event, RegistrationManager

annual_gala = Event(1, 'Annual Gala', 200)

mgr = RegistrationManager()
mgr.add(Registration(1, 'Taylor', annual_gala, 750))
mgr.add(Registration(2, 'Jordan', annual_gala, 150))
mgr.add(Registration(3, 'Morgan', annual_gala, 300))
mgr.add(Registration(4, 'Riley', annual_gala, 500))

mgr.registrations[0].confirm()
mgr.registrations[1].confirm()

mgr.report()
```

```
# tests/test_models.py
from models import Registration, Event, RegistrationManager

event = Event(1, 'Annual Gala', 200)

def test_default_status_is_pending():
    reg = Registration(1, 'Test', event, 100)
    assert reg.status == 'Pending'

def test_confirm_changes_status():
    reg = Registration(2, 'Test', event, 100)
    reg.confirm()
    assert reg.status == 'Confirmed'
```

```
def test_cancel_changes_status():
    reg = Registration(3, 'Test', event, 100)
    reg.cancel()
    assert reg.status == 'Cancelled'

def test_is_vip_true():
    reg = Registration(4, 'Test', event, 750)
    assert reg.is_vip() is True

def test_is_vip_false():
    reg = Registration(5, 'Test', event, 300)
    assert reg.is_vip() is False

def test_is_vip_boundary():
    reg = Registration(6, 'Test', event, 500)
    assert reg.is_vip() is False  # > not >=

def test_manager_add_increases_count():
    mgr = RegistrationManager()
    mgr.add(Registration(7, 'Test', event, 100))
    assert len(mgr.registrations) == 1

def test_manager_filter_by_status():
    mgr = RegistrationManager()
    r1 = Registration(8, 'A', event, 100)
    r2 = Registration(9, 'B', event, 200)
    mgr.add(r1); mgr.add(r2)
    r1.confirm()
    confirmed = mgr.list_by_status('Confirmed')
    assert len(confirmed) == 1
    assert confirmed[0].attendee_name == 'A'

def test_managers_are_independent():
    mgr1 = RegistrationManager()
    mgr2 = RegistrationManager()
    mgr1.add(Registration(10, 'A', event, 100))
    assert len(mgr2.registrations) == 0
```

9 Appendix B — Other Domain Ideas (for students stuck on Part 1)

If a student is genuinely stuck choosing a domain, these are quick, verified-workable options with a natural two-entity relationship and an obvious numeric total:

- **Library checkout system:** `Loan` (book, borrower, due_date, status) + `Book` (title, author) + `LoanManager`. Business rule: overdue if `days_out > 14`.
- **Gym membership manager:** `Membership` (member_name, plan, monthly_fee, status) + `Plan` (name, monthly_fee, perks) + `MembershipManager`. Business rule: premium if `monthly_fee > 50`.
- **Food delivery orders:** `Order` (customer, restaurant, total, status) + `Restaurant` (name, cuisine) + `OrderManager`. Business rule: requires driver bonus if `total > 75`.

Each maps cleanly onto the required template (two entity classes with a real relationship, a manager holding a list of the first, a numeric field for `total()`, and an obvious boundary-testable business rule) — offer these only to students who are stuck, not as a default to steer everyone toward, since genuine domain choice is part of the exercise's value.